Defining Accessors

In this lesson, we'll explore the usage of TypeScript accessor methods, specifically focusing on getters and setters. This will help us understand how we can access and modify private data within our classes.

Customer Class and Private Properties

Let's begin by examining our current setup. We have a `Customer` class, which contains private properties for the first name and last name. You may wonder how we can access this private data. This is where accessor methods come into play.

Traditional Getter and Setter Methods

In many programming languages, we use traditional methods to get and set private properties. TypeScript allows us to follow this very traditional approach, defining getter and setter methods.

Here's an example:

```
class Customer {
  private firstName: string;

  getFirstName(): string {
    return this.firstName;
  }

  setFirstName(name: string): void {
    this.firstName = name;
  }
}
```

File: Customer.ts

```
class Customer {

    private firstName: string;
    private lastName: string;
    … …

    public getFirstName(): string {
        return this.firstName;
    }

    public setFirstName(theFirst: string): void {
        this.firstName = theFirst;
    }
}
```

```
// now let's use it
let myCustomer = new Customer("Martin", "Dixon");

myCustomer.setFirstName("Greg");
console.log(myCustomer.getFirstName());
```

To use these methods, we would create an instance of the class, set the data, and then retrieve it:

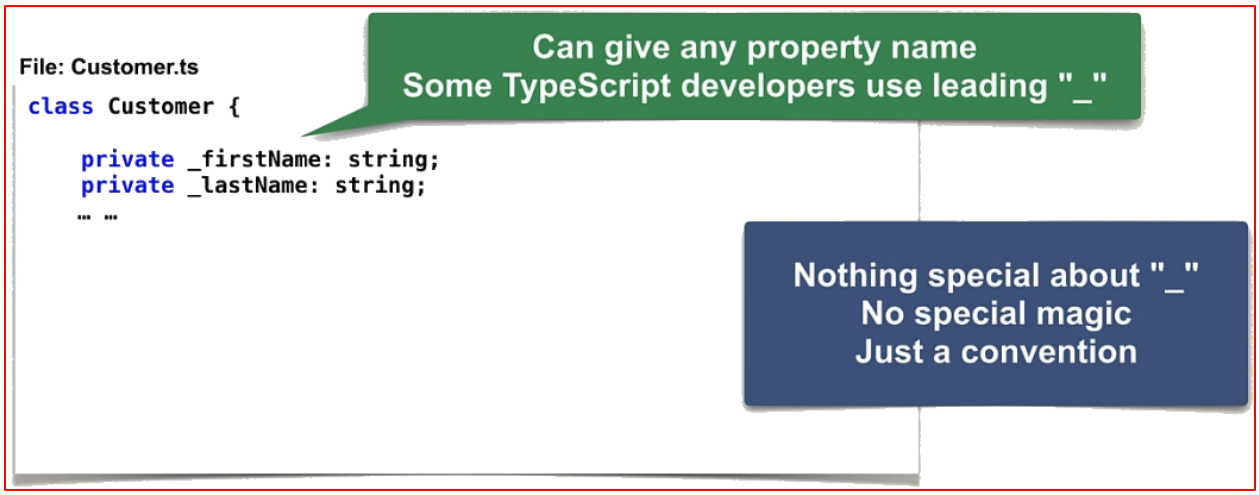
```
let myCustomer = new Customer();
myCustomer.setFirstName("Greg");
console.log(myCustomer.getFirstName()); // Output: Greg
```

TypeScript Accessors

While the traditional approach works, TypeScript offers a more succinct syntax using special get and set methods known as "accessors."

Changing Internal Property Names

First, we can change the internal property names. Although some developers use a leading underscore, there's no special significance to this convention; you can name the internal properties anything since they're private.



Defining Get and Set Accessors

Next, we define the accessor methods using the `get` and `set` keywords:

```
class Customer {
  private _firstName: string;

  get firstName(): string {
    return this._firstName;
  }

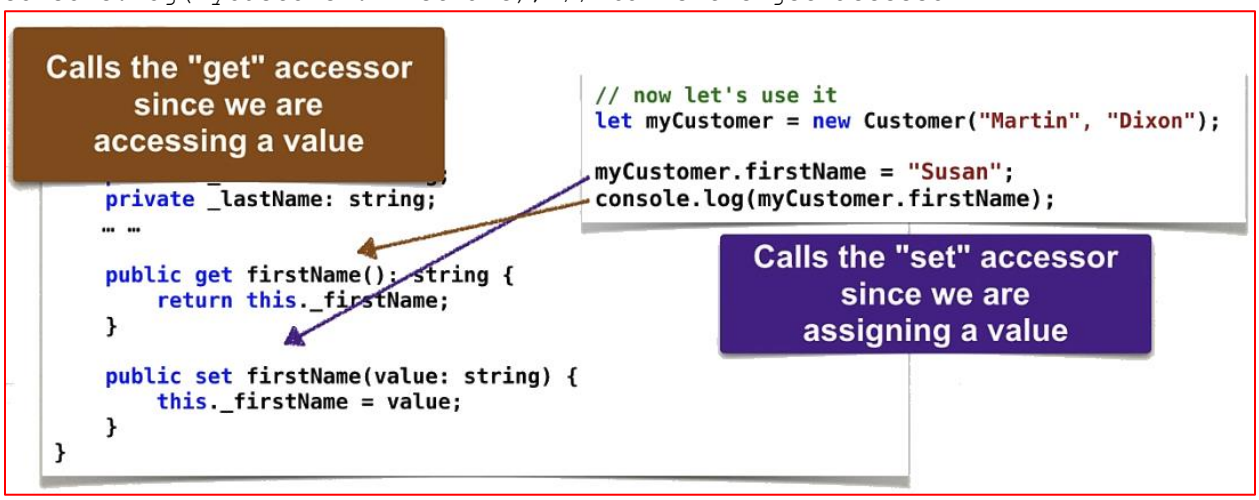
  set firstName(name: string) {
    this._firstName = name;
  }
}
```

Notice the absence of a return type for the `set` method; not even `void` is used.

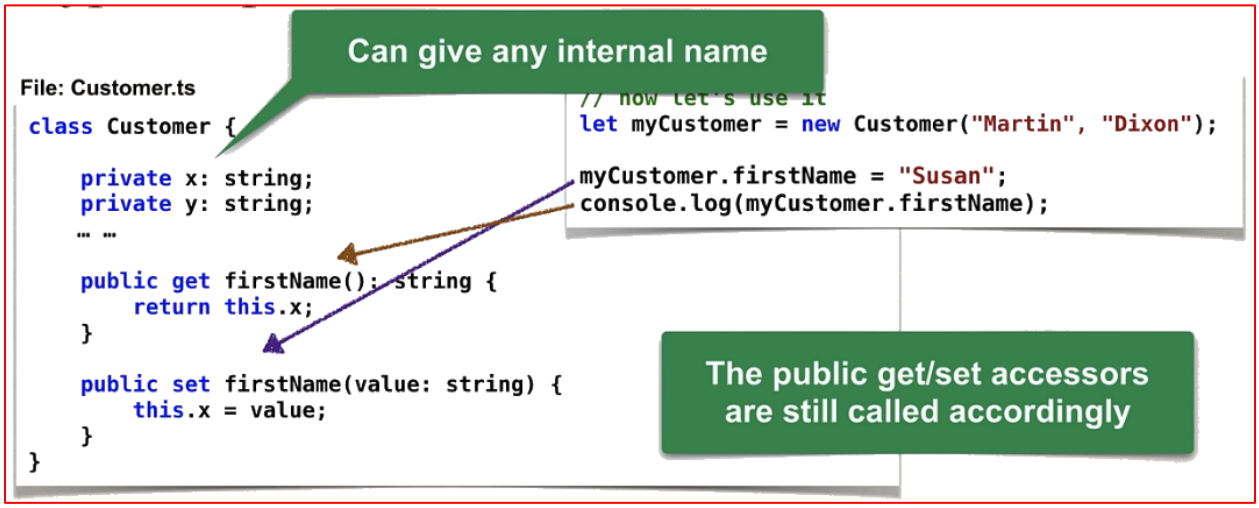
Using Accessors

With accessors, the syntax for accessing the data becomes more straightforward:

```
let myCustomer = new Customer();
myCustomer.firstName = "Susan"; // Calls the set accessor
console.log(myCustomer.firstName); // Calls the get accessor
```



The external code doesn't interact with the private fields directly. We can even rename the properties to unconventional names like `x` and `y`, and the public accessors will still work as expected.



Refactoring

After renaming our fields back to their original names and cleaning up our code a bit, we'll explore compiler flags.

Compiler Flags and Configuration

The get and set accessor feature is only supported in ES5 and higher. You'll need to manually set the compiler flag (`--target ES5`) when compiling with `tsc`.

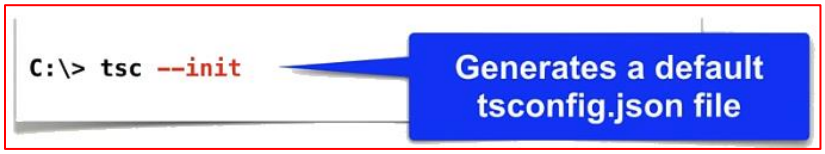


To simplify things, TypeScript offers a configuration file (`tsconfig.json`) to define compiler options and project settings. You can either create this file manually or generate it using the command `tsc --init`.

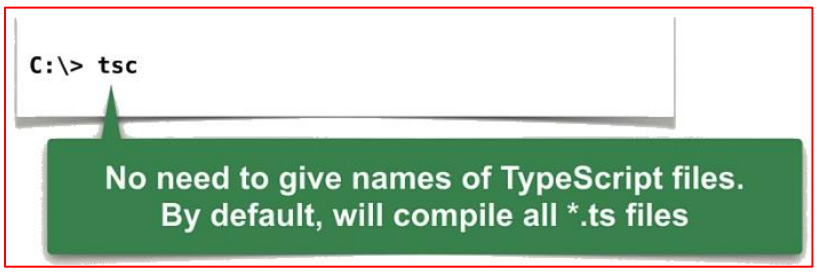
File: tsconfig.json

```
{
  "compilerOptions": {
    "noEmitOnError": true,
    "target": "es5"
  }
}
```

You can also generate a template for this file.



Then edit the `tsconfig.json` accordingly for your project requirements. Once your project has a `tsconfig.json` file, then you can compile it with `tsc` command.



Summary

Accessors in TypeScript provide an elegant way to handle private data within classes. Whether using traditional methods or the special `get` and `set` syntax, we can control the access and modification of private properties.

File: Customer.ts

```
class Customer {

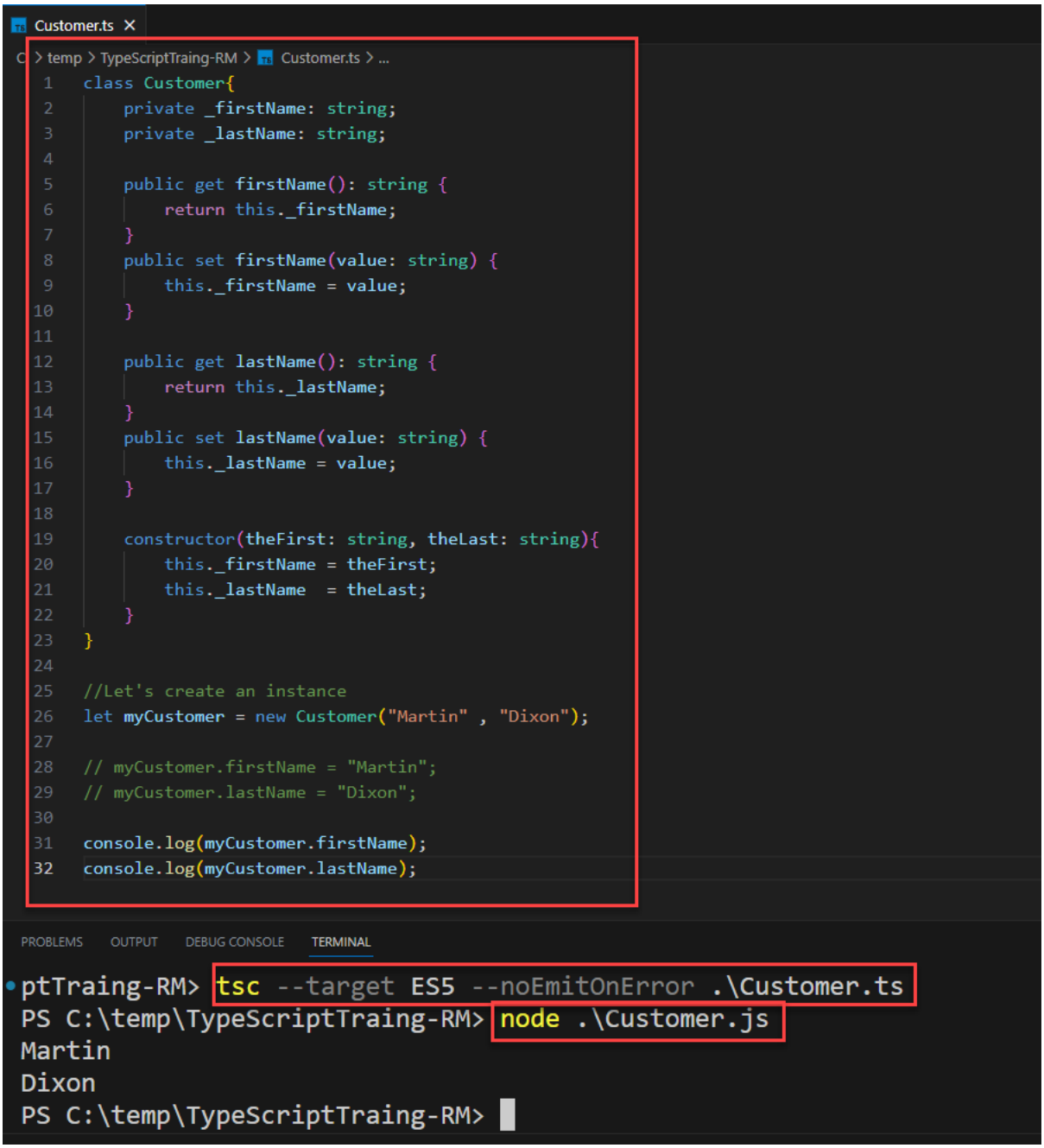
    private _firstName: string;
    private _lastName: string;
    … …

    get firstName(): string {
        return this._firstName;
    }

    set firstName(value: string) {
        this._firstName = value;
    }
}
```

```
// now let's use it
let myCustomer = new Customer("Martin", "Dixon");

myCustomer.firstName = "Susan";
console.log(myCustomer.firstName);
```


Customer.ts
temp > TypeScriptTraing-RM > Customer.ts > ...
class Customer{
private _firstName: string;
private _lastName: string;
public get firstName(): string {
return this._firstName;
}
public set firstName(value: string) {
this._firstName = value;
}
public get lastName(): string {
return this._lastName;
}
public set lastName(value: string) {
this._lastName = value;
}
constructor(theFirst: string, theLast: string){
this._firstName = theFirst;
this._lastName = theLast;
}
}
//Let's create an instance
let myCustomer = new Customer("Martin" , "Dixon");
// myCustomer.firstName = "Martin";
// myCustomer.lastName = "Dixon";
console.log(myCustomer.firstName);
console.log(myCustomer.lastName);
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
ptTraing-RM> tsc --target ES5 --noEmitOnError .\Customer.ts
PS C:\temp\TypeScriptTraing-RM> node .\Customer.js
Martin
Dixon
PS C:\temp\TypeScriptTraing-RM>